LAPORAN PRAKTIKUM
STRUKTUR DATA

ABSTRACT DATA TYPE

disusun oleh :

Khairunnisa M.

NIM 2511532005

Dosen Pengampu :

Dr. Wahyudi,. S.T., M.T.

Asisten Dosen :

M. Ghalid

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah yang sedang ditempuh.

Adapun laporan ini membahas mengenai implementasi konsep *Abstract Data Type* (ADT) menggunakan bahasa pemrograman Java, khususnya dalam pembuatan program yang merepresentasikan waktu (jam, menit, dan detik). Dalam proses penyusunan laporan ini, penulis memperoleh banyak pemahaman baru terkait penerapan konsep pemrograman berorientasi objek, seperti enkapsulasi, penggunaan method, serta interaksi antar objek.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat bagi pembaca dan dapat menjadi referensi dalam memahami materi yang dibahas.

Padang, 31 Maret 2026

Khairunnisa M.

DAFTAR ISI

KATA PENGANTAR ii

DAFTAR ISI iii

BAB I 1

PENDAHULUAN 1

1.1. Latar Belakang 1

1.2. Tujuan Praktikum 1

1.3. Manfaat Praktikum 2

BAB II 3

PEMBAHASAN 3

2.1. Konsep Abstract Data Type (ADT) dan OOP 3

2.2. Implementasi Class Jam 3

2.2.1 Deklarasi Class dan Atribut 3

2.2.2 Method Validasi 3

2.2.3 Konstruktor 4

2.2.4 Method Getter dan Setter 4

2.3. Operasi pada ADT Jam 4

2.3.1. Method Getter dan Setter 4

2.3.2. Operasi Relasional 5

2.3.3. Aritmatika 5

2.3.4. Operasi Durasi 5

2.3.5. Method Output (toString) 5

2.4. Implementasi Driver Program 6

2.4.1. Driver Program 1 (Jam Driver) 6

2.4.2. Driver Program 2 (Jam Driver2) 6

2.5. Operasi pada ADT Jam.....Error! Bookmark not defined.

BAB III **8**

KESIMPULAN DAN SARAN..... **8**

3.1 Kesimpulan..... **8**

3.2 Saran..... **8**

DAFTAR PUSTAKA **9**

Bibliography..... **9**

BAB I

PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi informasi menuntut adanya pemahaman yang baik terhadap konsep dasar pemrograman, khususnya dalam pengolahan data dan pembuatan sistem yang terstruktur. Salah satu konsep penting dalam pemrograman adalah *Abstract Data Type* (ADT), yang memungkinkan data direpresentasikan secara abstrak beserta operasi-operasi yang dapat dilakukan terhadapnya.

Dalam pemrograman berorientasi objek, ADT diimplementasikan dalam bentuk class yang memiliki atribut dan method. Konsep ini mempermudah pengembang dalam mengelola data serta meningkatkan keterbacaan dan modularitas program.

Pada praktikum ini, dilakukan implementasi ADT dalam bentuk program Java yang merepresentasikan waktu (jam, menit, dan detik). Program ini tidak hanya menyimpan data waktu, tetapi juga menyediakan berbagai operasi seperti perbandingan, penjumlahan, serta perhitungan durasi waktu. Selain itu, dibuat pula driver program untuk menguji fungsionalitas dari class yang telah dibuat.

1.2. Tujuan Praktikum

1. Memahami konsep Abstract Data Type (ADT) dalam pemrograman
2. Menerapkan konsep Object-Oriented Programming (OOP) dalam Java
3. Membuat classs yang merepresentasikan waktu beserta operasinya
4. Menguji program menggunakan driver sederhana dan interaktif

1.3. Manfaat Praktikum

1. Menambah pemahaman mengenai konsep ADT dan OOP
2. Melatih kemampuan dalam membuat program yang terstruktur dan modular
3. Memahami cara melakukan validasi dan manipulasi data dalam program
4. Menjadi dasar dalam pengembangan program yang lebih komplek.

BAB II

PEMBAHASAN

2.1. Konsep Abstract Data Type (ADT) dan OOP

Abstract Data Type (ADT) merupakan suatu konsep yang mendefinisikan tipe data berdasarkan operasi-operasi yang dapat dilakukan terhadap data tersebut, tanpa memperhatikan bagaiman implementasinya secara detail. Dalam pemrograman berorientasi objek OOP, ADT diimplementasikan dalam bentuk class yang memiliki atribut dan method.

Pada program ini, ADT direpresentasikan melalui class Jam_2511532005 yang digunakan untuk memodelkan waktu dalam satuan jam, menit, dan detik.

2.2. Implementasi Class Jam

2.2.1 Deklarasi Class dan Atribut

Class Jam_2511532005vmemiliki tiga atribut utama yaitu jam (hh), menit (mm), dan detik (ss). Ketiga atribut tersebut dibuat dengan akses private untuk menjaga keamanan data (enkapulasi).

```
public class Jam_2511532005 {
    private int hh; //0..23
    private int mm; //0..59
    private int ss; //0..59
```

Gambar 2.1 Deklarasi Class dan Atribut

2.2.2 Method Validasi

Method isValid() digunakan untuk memastikan bahwa nilai jam, menit, dan detik berada dalam rentang yang benar sebelum digunakan untuk membuat objek.

```
//---Validator---
public static boolean isValid(int h, int m, int s) {
    return (0 <= h && h <= 23) &&
           (0 <= m && m <= 59) &&
           (0 <= s && s <= 59);
}
```

Gambar 2.2 Method Validasi

2.2.3 Konstruktor

Konstruktor digunakan untuk menginisialisasi objek Jam_2511532005 saat pertama kali dibuat dengan nilai jam, menit, dan detik tertentu.

```
//--- Konstruktor/Kreator---
public Jam_2511532005(int h, int m, int s) {
    this.hh = h; this.mm = m; this.ss = s;
}
```

Gambar 2.3 Konstruktor

2.2.4 Method Getter dan Setter

Getter digunakan untuk mengambil nilai atribut, sedangkan setter digunakan untuk mengubah niali atribut.

```
//--- Selektor---
public int getHour() { return hh; }
public int getMinute() { return mm; }
public int getSecond() { return ss; }

//---Mutator---
public void setHour (int h) {this.hh = h; }
public void setMinutes (int m) { this.mm = m; }
public void setSecond (int s) { this.ss = s; }
```

Gambar 2.4 Getter dan Setter

2.3. Operasi pada ADT Jam

2.3.1. Method Getter dan Setter

Method toSeconds() digunakan untuk mengubah waktu menjadi satuan detik, sedangkan fromSeconds() digunakan untuk mengubah detik Kembali menjadi objek jam.

```java
//---Konversi---
public int toSeconds() { return hh * 3600 + mm * 60 + ss;}
public static Jam_2511532005 fromSeconds (int total) {
    if (total < 0) throw new IllegalArgumentException ("detik negatif");
    total %= 24 * 3600; //wrap 24 jam
    int h = total / 3600; total %= 3600;
    int m = total / 60; int s = total % 60;
    return new Jam_2511532005(h,m, s);
}
```

Gambar 2.5 Konversi Waktu.

2.3.2. Operasi Relasional

Operasi relasional digunakan untuk membandingkan dua objek jam.

```java
//---relasional---
public int compareTo(Jam_2511532005 other) {return Integer.compare(this.toSeconds(), other.toSeconds()); }
public boolean equals(Object o) {
    if (!(o instanceof Jam_2511532005 j)) return false;
    return hh == j.hh && mm == j.mm && ss == j.ss;
}
public int hashCode( ) { return java.util.Objects.hash(hh, mm, ss); }
```

Gambar 2.6 Operasi Relasional

2.3.3. Aritmatika

Operasi ini digunakan untuk melakukan manipulasi waktu seperti penjumlahan, pengurangan, serta penambahan detik.

```java
//---Aritmatika---
public Jam_2511532005 plus(Jam_2511532005 other) {return fromSeconds(this.toSeconds() + other.toSeconds()); }
public Jam_2511532005 minus(Jam_2511532005 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600)); }
public Jam_2511532005 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
public Jam_2511532005 nextNSeconds(int n) {return fromSeconds(this.toSeconds() + Math.max(0, n));}
public Jam_2511532005 prevSeconds() {return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600)); }
public Jam_2511532005 prevNSeconds(int n) {return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600 )); }
```

Gambar 2.7 Operasi Aritmatika.

2.3.4. Operasi Durasi

Method durasiDetik() digunakan untuk menghitung selisih waktu dalam satuan detik.

```java
//---Durasi Detik---bisa negatif jika this > other? sesuai spesifikasi durasi (JAw, JAkh)
public static int durasiDetik (Jam_2511532005 jaw, Jam_2511532005 jakh) {return jakh.toSeconds() - jaw.toSeconds(); }
```

Gambar 2.8 Operasi Durasi.

2.3.5. Method Output (toString)

Method toString() digunakan untuk menampilkan objek jam dalam format string.

```java
    //---overriding ke string
    public String toString() {return String.format("%02d:%02d;%02d", hh, mm, ss); }

}
```

Gambar 2.9 Method Output.

2.4. Implementasi Driver Program

2.4.1. Driver Program 1 (Jam Driver)

Program ini digunakan untuk menguji fungsi-fungsi pada class Jam tanpa menggunakan input dari pengguna.

```java
package pekan1_2511532005;

public class JamDriver_2511532005 {
    public static void main(String[] args) {
        Jam_2511532005 a = new Jam_2511532005(23, 59, 50);
        Jam_2511532005 b = new Jam_2511532005(0, 0, 15);
        System.out.println("a                 =   " + a);
        System.out.println("b                 =   " + b);
        System.out.println("a+b               = " + a.plus(b));
        System.out.println("next 20s          = " + a.nextNSeconds(20));
        System.out.println("durasi(a,b)       = " + Jam_2511532005.durasiDetik(a, b));
        System.out.println("a.compareTo(b)    = " + a.compareTo(b));
    }

}
```

Gambar 2.9 Driver Program 1

Program ini melakukan beberapa operasi seperti:

- Penjumlahan waktu
- Penambahan detik
- Perbandingan waktu
- Perhitungan durasi

2.4.2. Driver Program 2 (Jam Driver2)

Program ini menggunakan input dari pengguna untuk membuat objek jam dan melakukan berbagai operasi. Program ini menggunakan Scanner untuk membaca data jam, menit, dan detik, kemudian memvalidasi input melalui methoc isValid() dalam perulangan hingga diperoleh nilai yang benar. Program kemudian melakukan beberapa operasi seperti perbandingan waktu (compareTo()), perhitungan durasi (durasiDetik()), serta manipulasi waktu (nextSecond(), prevSeconds(), dan plus()), sehingga menunjukkan penerapan konsep input/output, validasi, percabangan, dan penggunaan method dalam ADT.

```java
package pekan1_2511532005;
import java.util.Scanner;

public class JamDriver2_2511532005 {
    public static void main(String[] args) {
        Scanner input = new Scanner (System.in);
        System.out.println("=== Program Driver Objek Jam ===");
        // 1. input jam pertama
        System.out.println("\n---Input Jam 1---");
        Jam_2511532005 j1 = buatJamDariInput(input);

        //2. input Jam Kedua
        System.out.println("\n---Input Jam 2---");
        Jam_2511532005 j2 = buatJamDariInput(input);

        //3.Menampilkan Data
        System.out.println("\n---Hasil Operasi---");
        System.out.println("Jam 1 (String)          :" + j1.toString());
        System.out.println("Jam 2 (String)          :" + j2.toString());
        System.out.println("Jam 1 dalam detik       :" + j1.toString());
        System.out.println("Jam 2 dalam detik       :" + j1.toString());

        //4. Operasional Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status              : Jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
            System.out.println("Status              : Jam 1 lebih awal (sebelum) Jam 2");
        } else {
            System.out.println("Status              : Jam 1 dan Jam 2 sama persis");
        }

        //5. Operasi aritmatika
        System.out.println("Durasi (J1 ke J2) : " + Jam_2511532005.durasiDetik(j1, j2) + "detik");

        Jam_2511532005 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik berikutnya: " + jNext);

        Jam_2511532005 jPrev = j1.prevSeconds();
        System.out.println("Jam 1 Detik sebelumnya: " + jPrev);
```

```java
        //6. Operasi Penjumlahan Jam
        Jam_2511532005 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2 : " + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");
    }
    /**
     * Prosedur pembantu untuk melakukan input dan validasi secara berlang
     * sampai user memasukkan angka yang benar (0..23, 0..59).
     */

    private static Jam_2511532005 buatJamDariInput(Scanner sc) {
        int h, m, s;
        while (true) {
            System.out.print("Masukkan Jam (0-23) : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59) : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt();

            //Memanggil method static isValid dari kelas Jam
            if (Jam_2511532005.isValid(h, m, s)) {
                return new Jam_2511532005(h, m, s);
            } else {
                System.out.println("[Error] Input tidak Valid! Silahkan ulangi. \n");
            }
        }
    }

}
```

Gambar 2.10 Driver Program 2.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi *Abstract Data Type* (ADT) dalam bahasa pemrograman Java dapat dilakukan dengan menggunakan class yang berisi atribut dan method. Pada program yang dibuat, class Jam_2511532005 berhasil merepresentasikan data waktu dalam bentuk jam, menit, dan detik serta menyediakan berbagai operasi seperti konversi, perbandingan, penjumlahan, dan perhitungan durasi.

Selain itu, penggunaan driver program, baik yang sederhana maupun interaktif, membantu dalam menguji fungsionalitas class serta memberikan gambaran penerapan program dalam kondisi nyata. Program ini juga telah menerapkan beberapa konsep dasar pemrograman seperti enkapsulasi, validasi data, percabangan, dan perulangan.

Dengan demikian, praktikum ini memberikan pemahaman yang baik mengenai penerapan ADT dan OOP dalam pembuatan program yang terstruktur dan modular.

3.2 Saran

Adapun saran untuk pengembangan program selanjutnya adalah:

1. Menambahkan validasi pada method setter agar data lebih aman
2. Memperbaiki format output agar lebih konsisten
3. Mengembangkan program dengan fitur tambahan seperti input format waktu otomatis
4. Mengintegrasikan program dengan antarmuka grafis (GUI) agar lebih interaktif

DAFTAR PUSTAKA

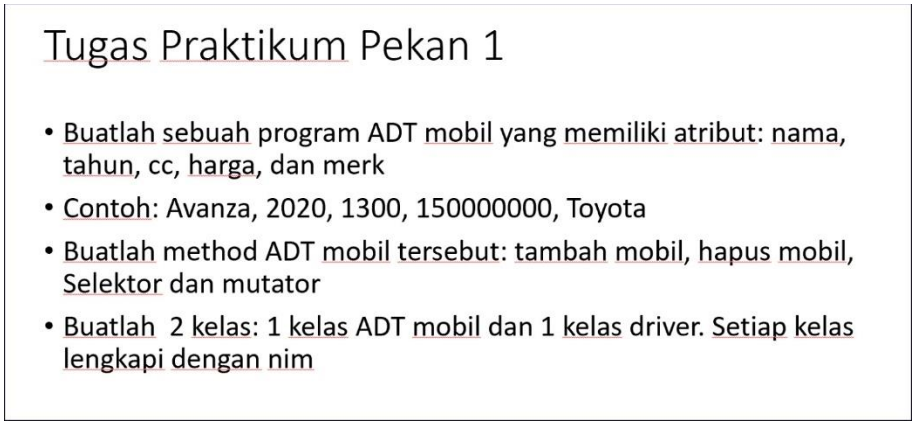
Bibliography

Abstract Data Type. (n.d.). Retrieved from NUS Computing: https://www.comp.nus.edu.sg/~cs1020/lect/15s2/Lect9-ADT.pdf

Ramesh, A. (2019, October 18). *ADT Java Tutorial*. Retrieved from Java Code Geeks: https://examples.javacodegeeks.com/adt-java-tutorial/

TUGAS PRAKTIKUM 1 ADT

1. Perintah Tugas



Gambar 1.1 Perintah Tugas Praktikum Pekan .

2. Kode Program Mobil_2511532005

Berikut Kode Program Mobil_2511532005 yang merupakan implementasi dari Abstract Data Type (ADT) yang digunakan untuk mempresentasikan data sebuah mobil.

```java
package pekan1_2511532005;

public class Mobil_2511532005 {
    private String nama;
    private int tahun;
    private int cc;
    private long harga;
    private String merk;

    public Mobil_2511532005(String nama, int tahun, int cc, long harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }
    public String getNama() { return nama; }
    public int getTahun() { return tahun; }
    public int getCc() { return cc; }
    public long getHarga() { return harga; }
    public String getMerk() { return merk; }

    // Mutator (setter)
    public void setNama(String nama) { this.nama = nama; }
    public void setTahun(int tahun) { this.tahun = tahun; }
    public void setCc(int cc) { this.cc = cc; }
    public void setHarga(long harga) { this.harga = harga; }
    public void setMerk(String merk) { this.merk = merk; }

    public void tambahMobil(String nama, int tahun, int cc, long harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }
```

```java
    public void hapusMobil() {
        this.nama = "";
        this.tahun = 0;
        this.cc = 0;
        this.harga = 0;
        this.merk = "";
    }
    public String toString() {
        return nama + " | " + tahun + " | " + cc + "cc | Rp" + harga + " | " + merk;
    }
```

Gambar 2.1 Kode Program Mobil_2511532005.

Penjelasan fungsi atau kegunaan dari kode program ini :

a. Deklarasi Class, berfungsi sebagai driver (program utama) untuk menjalankan dan menguji class mobil.
b. Pembuatan Objek, Dimana di kode program Mobil_2511532005 tertulis dalam bentuk (a,b).
c. Menampilkan objek, menggunakan toString().
d. Mutator (Setter), untuk dapat mengubah harga mobil.
e. Selektor (Getter), untuk mengambil nilai atribut menggunakan getter.
f. Method tambahMobil, untuk mengganti seluruh data mobil a dengan data baru.
g. Method hapusMobil, untuk menghapus data mobil b dengan mereset atau menghapus semua atribut.

3. Kode Program MobilDriver_2511532005

```java
package pekan1_2511532005;

public class MobilDriver_2511532005 {
    public static void main(String[] args) {
        Mobil_2511532005 a = new Mobil_2511532005("Avanza", 2020, 1300, 150000000, "Toyota");
        Mobil_2511532005 b = new Mobil_2511532005("Brio", 2022, 1200, 180000000, "Honda");

        System.out.println("a = " + a);
        System.out.println("b = " + b);

        a.setHarga(160000000);
        System.out.println("a setelah ubah harga = " + a);

        System.out.println("Nama mobil a = " + a.getNama());
        System.out.println("Merk mobil b = " + b.getMerk());

        a.tambahMobil("Xenia", 2021, 1300, 155000000, "Daihatsu");
        System.out.println("a setelah tambahMobil = " + a);

        b.hapusMobil();
        System.out.println("b setelah hapus = " + b);
    }
}
```

Gambar 3.1 Kode Program MobilDriver_2511532005.

Penjelasan fungsi atau kegunaan dari kode program ini :

a. Deklarasi Class, berfungsi sebagai mereprasentasikan objek mobil dengan atribut tertentu.

b. Atribut, ada beberapa atribut yang digunakan di dalam kode program. Setiap atribut dilengkapi dengan keyword private agar data tidak bisa diaksis langsung dari luar class
c. Konstruktor, untuk inisialisasi objek saat pertamakali dibuat.
d. Mutator (Setter), untuk dapat mengambil inisialisasi objek saat pertama kali dibuat.
e. Selektor (Getter), untuk mengambil nilai atribut dari objek.

4. Output Dari Kode Program ADT Mobil

```
a = Avanza | 2020 | 1300cc | Rp150000000 | Toyota
b = Brio | 2022 | 1200cc | Rp180000000 | Honda
a setelah ubah harga = Avanza | 2020 | 1300cc | Rp160000000 | Toyota
Nama mobil a = Avanza
Merk mobil b = Honda
a setelah tambahMobil = Xenia | 2021 | 1300cc | Rp155000000 | Daihatsu
b setelah hapus =  | 0 | 0cc | Rp0 |
```

Gambar 3.1 Output dari Kode Program ADT Mobil.